

AN
INTERNSHIP
REPORT
ON
“SALESSIGHT 360”

submitted in partial fulfilment of the requirements for awarding the degree of

Master of Computer Applications

(Session: 2025-27)

By:

Shaivya Aggarwal

(Roll No.: 2025215595)

Under the supervision of:

Dr. Kanwal Garg

Associate Professor



Department of Computer Science & Applications

Kurukshetra University, Kurukshetra

(August 2026)

Students' Day Diary

Day	Date	Detail of Learning Activity
1	11/06/2026	Introduction to SalesSight360 and understanding its objectives, scope, and requirements.
2	12/06/2026	Studied the available sales, customer, product, payment, marketing, CRM, and target datasets and understood their structure and purpose.
3	13/06/2026	Learned data exploration using Python and Pandas and inspected data types, missing values, and duplicates.
4	15/06/2026	Performed data cleaning and handled missing, duplicate, and inconsistent records.
5	17/06/2026	Cleaned and standardized customer, product, orders, and payment datasets.
6	19/06/2026	Worked on CRM and marketing data cleaning and identified important analytical fields.
7	20/06/2026	Learned data transformation techniques and prepared datasets for analytical processing.
8	22/06/2026	Studied relationships between customers, products, orders, and payments and designed the analytical data structure.
9	24/06/2026	Created dimension tables for customers and products as part of the analytical data model.
10	26/06/2026	Created fact tables for orders and payments and performed relationship validation.
11	27/06/2026	Prepared CRM and marketing analytical tables and standardized their structures.
12	29/06/2026	Performed data-model validation and checked for invalid customer, product, and order references.
13	30/06/2026	Calculated major sales KPIs including total sales, profit, orders, and average order value.
14	02/07/2026	Calculated profitability metrics such as net sales, discount amount, and profit margin.
15	04/07/2026	Performed marketing and CRM analysis using leads, conversions, campaign spend, deal value, and expected revenue.
16	06/07/2026	Prepared target-versus-actual sales analysis and calculated sales variance and target achievement.
17	08/07/2026	Performed monthly sales analysis and studied sales trends using order-date information.
18	10/07/2026	Analyzed product and customer performance to identify top-performing products and customers.
19	11/07/2026	Analyzed marketing campaign performance and compared campaign effectiveness.
20	13/07/2026	Started Power BI implementation and imported the prepared analytical datasets.
21	15/07/2026	Created table relationships and developed KPI cards for the SalesSight360 dashboard.

22	17/07/2026	Developed the Sales Performance dashboard with charts, KPIs, and interactive visuals.
23	18/07/2026	Developed marketing and customer insights visualizations in Power BI.
24	20/07/2026	Added slicers, filters, and navigation features to improve dashboard interactivity.
25	22/07/2026	Improved dashboard layout, formatting, labels, and overall visual presentation.
26	24/07/2026	Validated Power BI dashboard results against the Python/Pandas analysis and tested interactive features.
27	28/07/2026	Prepared project documentation, screenshots, methodology, and explanation of the SalesSight360 workflow.
28	30/07/2026	Completed the final project review and summarized the technical learning and outcomes of SalesSight360.

DECLARATION

I, **Shaivya Aggarwal**, bearing Roll Number **2025215595**, hereby declare that the project report titled **“SALESIGHT-360”**, submitted in partial fulfillment of the requirements for the award of the **MASTER OF COMPUTER APPLICATIONS** course is my original work and has been carried out under the guidance of **DR. Kanwal Garg**.

I also declare that this work has not been submitted previously by me or any other individual for the award of any degree, diploma, or certificate at this or any other institution.

Any references and resources used during the development of this project have been duly acknowledged in the report.

Shaivya Aggarwal



Department of Computer Science and Applications

KURUKSHETRA UNIVERSITY KURUKSHETRA

(Established by the State Legislature Act XII of 1956)

(‘A++’ Grade, NAAC Accredited)

Dr. Kanwal Garg

Email: kanwalgarg@kuk.ac.in

Date:14/08/2026

CERTIFICATE

This is to certify that the project report titled “**SALESIGHT 360**” submitted by Shaivya Aggarwal, University roll no. **2025215595** and Class roll no. **98**, in partial fulfillment of the requirements for the award of the **MASTER OF COMPUTER APPLICATIONS**, is a bona fide record of original work carried out by his under my supervision during the academic session 2025-2027.

This work has not been submitted previously for the award of any degree, diploma, or certificate in this or any other university/institute.

Dr. Kanwal Garg

(Associate Professor)



Department of Computer Science and Applications

KURUKSHETRA UNIVERSITY KURUKSHETRA

(Established by the State Legislature Act XII of 1956)

(‘A++’ Grade, NAAC Accredited)

Dr. Pardeep Kumar

Email: pkmittal@kuk.ac.in

Chairperson

Date:14/08/2026

CERTIFICATE

This is to certify that the project report entitled “**SALESIGHT 360**” is a bona fide work carried out by Raghav, University roll no. **2025215595** and Class roll no. **98** student of MASTER OF COMPUTER APPLICATIONS, under the guidance and supervision of **Dr. Kanwal Garg**, Department of Computer Science and Applications, Kurukshetra University, Kurukshetra.

The project work was conducted at under DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS during the period from **11-06-2026 to 30-07-2026**. This project is approved for submission toward the partial fulfillment of the requirements for the award of the degree of MASTER OF COMPUTER APPLICATIONS.

Dr. Pardeep Kumar

Chairperson

Department of Computer Science and Applications

Kurukshetra University, Kurukshetra

Date: 14/08/2026

ACKNOWLEDGEMENT

Acknowledgement is the perfect way to convey heartiest thanks to all outstanding personalities. I consider myself fortunate to get the opportunity of doing a project, yet the opportunity could not have been utilized without the guidance and support of many individuals.

I extend my sincere gratitude to all those who have played a pivotal role in the successful completion of this project.

Firstly, I am deeply thankful to my guide **Dr. Kanwal Garg**, Associate Professor in the Department of Computer Science and Applications, Kurukshetra University, Kurukshetra for his unwavering guidance and mentorship. His insights, feedback, and patient explanations have been invaluable, and his commitment to academic excellence has propelled this project forward.

I am also thankful to **Dr. Pardeep Kumar**, Professor and Chairperson in the Department of Computer Science and Applications, Kurukshetra University, Kurukshetra and other faculty members.

At last, I would like to extend my heartfelt gratitude to my parents and friends for their unwavering support and encouragement throughout this journey.

To all, I offer my heartfelt thanks for being part of this journey, and I am humbled by the combined impact of your guidance, expertise, and divine blessings.

(Shaivya Aggarwal)

INDEX

TOPIC		Page No.
Chapter- 1	Introduction	8-12
	1.1 About the Project	
	1.2 Objectives of the Project	
	1.3 Relevance of the Project	
	1.4 Scope of the Project	
	1.5 Problem Statement	
Chapter- 2	Development Environment & Technologies	13-23
	2.1 Development Environment	
	2.2 Software Requirements	
	2.3 Hardware Requirements	
	2.4 Programming Language Used	
	2.5 Technologies and Libraries Used	
	2.6 Development Tools Used	
	2.7 Why These Technologies Were Selected	
Chapter- 3	Analysis Document	24-32
	3.1 Introduction	
	3.2 Functional Requirement	
	3.3 Data Flow Diagram (DFD Level 0)	
	3.4 Data Flow Diagram (DFD Level 1)	
	3.5 Analysis Summary	
Chapter- 4	Design Document	33-38

	4.1	Introduction	
	4.2	Overall System Architecture	
	4.3	Modular Design	
	4.4	Database schema	
	4.5	Data Integrity Constraints	
	4.6	Module Interaction	
	4.7	User Interface Design	
	4.	Summary	
Chapter- 5	Program Code		39-84
	5.1	Introduction	
	5.2	Main Module	
	5.3	Data Module	
	5.4	Customer Analysis	
	5.5	Product Performance Analysis	
	5.6	Sales trend analysis	
	5.7	Target Versus Actual Analysis	
	5.8	Marketing and campaign analysis	
	5.9	CRM and pipeline analysis and KPI Calculation and Analysis	
Chapter-6	Testing		85-86
	6.1	Introduction	
	6.2	Testing Methodology	
	6.3	Test Cases	
	6.4	Testing Summary	
Chapter- 7	Limitations of the Project		87-88
	7.1	Introduction	
	7.2	Limitations	

Chapter- 8	Conclusion And Future Scope		89-94
	8.1	Introduction	
	8.2	Future Scope	
	8.3	Applications of the Project	
	8.4	Benefits of future enhancements	
	8.5	Summary	
Chapter- 9	Bibliography		95
	Official Documentation		
	Software and Development Tools		

CHAPTER1

INTRODUCTION

1.1 About the Project

SalesSight 360 – Multi-Channel Revenue Analytics Platform is a data analytics and business intelligence project designed to provide a unified view of an organization's sales and revenue-related activities. In modern organizations, business data is generated from multiple sources such as sales transactions, customer information, payments, marketing campaigns, Customer Relationship Management (CRM) systems, and sales targets. When this information is maintained separately, it becomes difficult to analyse overall business performance and make data-driven decisions.

SalesSight 360 addresses this challenge by integrating data from multiple business areas into a structured analytical environment. The project uses **Python and Pandas** for data extraction, cleaning, transformation, validation, and preparation. The processed data is organized into analytical tables that can be used for consistent reporting and analysis. **Microsoft Power BI** is then used to create interactive dashboards and visualizations for presenting important business insights.

The main purpose of SalesSight 360 is to transform raw and disconnected business data into meaningful information that can help users understand business performance more effectively. By combining data preparation, analytical modelling, KPI calculation, and interactive visualization, the project demonstrates how data analytics can support better business understanding and decision-making. Overall, SalesSight 360 provides an end-to-end analytics workflow starting from raw datasets and ending with interactive business intelligence dashboards. The project demonstrates the practical application of **ETL, data cleaning, data validation, data modelling, exploratory analysis, KPI development, and data visualization** in a real-world business analytics scenario.

1.2 Objectives of the Project

The main objective of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** is to develop an integrated data analytics solution that transforms raw business data into meaningful insights for monitoring and evaluating business performance. The specific objectives of the project are:

1. **To integrate multiple business datasets** such as customers, products, orders, payments, marketing, CRM, and sales target data into a unified analytical environment.

2. **To perform data cleaning and preprocessing** using Python and Pandas to handle inconsistencies, missing values, data types, duplicate records, and other data-quality issues.
3. **To develop an ETL (Extract, Transform, Load) process** for converting raw datasets into clean, structured, and analysis-ready data.
4. **To create a reliable analytical data model** using dimension and fact-style tables to support efficient analysis and reporting.
5. **To calculate important business KPIs** such as total sales, total profit, average order value, profit margin, conversion rate, cost per lead, deal value, and target achievement.
6. **To analyse sales performance** based on factors such as time, products, customers, and other relevant business dimensions.
7. **To evaluate marketing performance** by analysing leads, conversions, marketing expenditure, conversion rates, and campaign effectiveness.
8. **To analyse CRM data** in order to understand deal value and expected revenue within the sales pipeline.
9. **To perform target-versus-actual analysis** to identify sales variance and measure the level of target achievement.
10. **To develop interactive Power BI dashboards** that present business information through charts, KPI cards, filters, and other visualizations.
11. **To identify meaningful trends and patterns** from the available business data that can support data-driven decision-making.
12. **To provide a scalable analytics foundation** that can be further enhanced with forecasting, customer segmentation, automated data refresh, and advanced analytics techniques.

1.3 Relevance of the Project

In today's data-driven business environment, organizations generate large amounts of data from multiple sources such as sales transactions, customer interactions, marketing campaigns, payments, and Customer Relationship Management (CRM) systems. However, collecting data alone is not sufficient; organizations need to transform this data into meaningful information that can support effective decision-making. Therefore, data analytics and business intelligence have become important tools for monitoring business performance and identifying opportunities for improvement.

The **SalesSight 360 – Multi-Channel Revenue Analytics Platform** is relevant because it provides an integrated approach to analysing business data from different functional areas. Instead of analysing sales, marketing, customer, CRM, and target data separately, the project brings these datasets together into a structured analytical environment. The project is particularly relevant to the field of **Data Analytics and Business Intelligence** because it demonstrates an end-to-end process of extracting, cleaning, transforming, validating, modelling, and visualizing data. Python and Pandas are used for data preparation and ETL operations, while Power BI is used to convert the prepared data into interactive dashboards and reports.

SalesSight 360 also helps in evaluating important business performance indicators such as total sales, profit, orders, average order value, marketing conversion rate, cost per lead, CRM deal value, expected revenue, and target achievement. These insights can help users understand current business performance, identify trends and performance gaps, and make more informed decisions. The project is also relevant from an academic and practical perspective. Academically, it provides practical implementation of concepts such as **ETL, data preprocessing, data modelling, KPI analysis, exploratory data analysis, and data visualization**. From a practical perspective, it demonstrates how multiple business datasets can be transformed into a centralized analytical solution that supports performance monitoring and decision-making. Thus, SalesSight 360 is relevant as a practical application of modern data analytics techniques for converting raw, multi-source business data into meaningful and actionable business insights.

1.4 Scope of the Project

The scope of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** covers the collection, integration, processing, analysis, and visualization of business data from multiple functional areas. The project is designed to provide a consolidated view of business performance by bringing together sales, customer, product, payment, marketing, CRM, and target-related information. The major areas covered within the scope of the project are as follows:

1. **Data Integration:** The project integrates multiple datasets including customer, product, order, payment, marketing, CRM, and sales target data into a unified analytical environment.
2. **Data Cleaning and Preprocessing:** Raw data is processed using Python and Pandas to improve data quality, handle inconsistencies, convert data types, prepare dates, and create analysis-ready datasets.
3. **ETL Process:** An ETL workflow is implemented to extract data from source files, transform it according to analytical requirements, and prepare structured datasets for reporting.
4. **Data Modelling:** The project creates dimension and fact-style analytical tables such as **DIM_CUSTOMERS, DIM_PRODUCTS, FACT_ORDERS, FACT_PAYMENTS, FACT_CRM, and FACT_MARKETING** to support organized and consistent analysis.

5. **Sales Analysis:** The project analyses sales revenue, profit, orders, average order value, discount amount, net sales, monthly sales trends, product performance, and customer performance.
6. **Marketing Analysis:** Marketing data is analysed using metrics such as leads, conversions, marketing expenditure, conversion rate, cost per lead, and cost per conversion to evaluate campaign effectiveness.
7. **CRM Analysis:** CRM information is used to analyse total deal value and expected revenue, providing insights into the sales pipeline and potential future revenue.
8. **Target and Performance Analysis:** Actual sales are compared with predefined sales targets to calculate sales variance and target achievement percentage.
9. **Business Intelligence Dashboard:** The processed data is visualized using Power BI dashboards containing KPI cards, charts, trends, comparisons, filters, and interactive visualizations.
10. **Data Validation:** Data-quality checks are performed to verify relationships between important entities, including customer, product, and order identifiers.
11. **Decision Support:** The final dashboards and analytical outputs are intended to help users understand business performance, identify trends and gaps, and support data-driven decision-making.

Future Scope

The project can be further extended by incorporating **automated data refresh, real-time analytics, sales forecasting, machine learning-based prediction, customer segmentation, churn analysis, advanced marketing attribution, automated KPI alerts, and cloud-based deployment.**

Therefore, the scope of SalesSight 360 extends from basic data preparation to business intelligence reporting and provides a foundation for developing a more advanced revenue intelligence platform in the future.

1.5 Problem Statement

In modern organizations, business information is generated continuously from different operational areas such as sales transactions, customers, products, payments, marketing campaigns, Customer Relationship Management (CRM) systems, and sales targets. This information is often stored in separate datasets with different structures and formats. Analysing these sources independently makes it difficult to obtain a complete and consistent view of overall business performance.

Another major challenge is that raw business data is generally not suitable for direct analysis. It may require data cleaning, transformation, validation, and integration before meaningful insights can be generated. Without a structured ETL process and appropriate data modelling, the reliability and consistency of analytical results can be affected. Therefore, there is a need for an integrated analytics solution that can collect data from

multiple business sources, clean and transform it, validate relationships between datasets, organize it into an analytical data model, and present important business metrics through interactive visualizations.

The **SalesSight 360 – Multi-Channel Revenue Analytics Platform** is developed to address these challenges. The project integrates sales, customer, product, payment, marketing, CRM, and target data and transforms it into analysis-ready information using Python and Pandas. The prepared data is then used in Power BI to create interactive dashboards for analysing sales performance, profitability, marketing effectiveness, CRM pipeline, and target achievement.

CHAPTER 2

DEVELOPMENT ENVIRONMENT AND TECHNOLOGIES USED

2.1 Development Environment

The development environment refers to the combination of hardware, operating system, software applications, programming tools, libraries, and business intelligence platforms used for designing and implementing the **SalesSight 360 – Multi-Channel Revenue Analytics Platform**.

SalesSight 360 is developed as an end-to-end data analytics solution in which raw business datasets are processed and converted into meaningful analytical information. The development environment supports all major stages of the project, including data ingestion, data cleaning, data transformation, data validation, analytical modelling, KPI calculation, and interactive data visualization.

The project primarily uses **Python with the Pandas library** for data preparation and ETL operations. Python provides a flexible programming environment for handling structured datasets and performing data-processing operations. Pandas is used extensively for reading datasets, inspecting data, handling data inconsistencies, converting and creating required fields, merging related datasets, performing calculations, and generating analysis-ready tables.

Visual Studio Code (VS Code) is used as the primary development environment for writing and executing the Python scripts. It provides an organized workspace for maintaining the project files, executing the ETL program, viewing outputs, identifying errors, and modifying the data-processing workflow.

After the data preparation stage, **Microsoft Power BI** is used as the business intelligence and visualization platform. The processed datasets are connected to Power BI and used to develop interactive dashboards. These dashboards present important business measures such as sales, profit, orders, marketing conversions, CRM deal value, expected revenue, and target achievement through KPI cards, charts, tables, and interactive filters.

The project uses **CSV files as the primary source format** for storing the raw datasets. Separate datasets are maintained for customers, products, orders, payments, marketing, CRM, and sales targets. During the ETL process, these datasets are transformed into structured analytical tables such as **DIM_CUSTOMERS, DIM_PRODUCTS, FACT_ORDERS, FACT_PAYMENTS, FACT_CRM, and FACT_MARKETING**.

The development environment therefore provides a complete pipeline from raw data to business intelligence reporting. The overall workflow implemented in the project is:

Source CSV Files → Python/Pandas → Data Cleaning → Data Transformation → Data Validation → Analytical Data Model → Power BI → Interactive Dashboards → Business Insights

2.2 Software Requirements

The software requirements define the operating system, programming environment, libraries, development tools, data-storage formats, and business intelligence software required for the development and execution of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform**. The project requires a combination of Python-based data-processing tools and Microsoft Power BI for analytical reporting and visualization. The software requirements used in this project are listed below.

<i>Software</i>	<i>Purpose</i>
Windows 11	Operating System used for project development.
Python 3.13	Main programming language used for implementing the complete project.
Visual Studio Code	Code editor used for writing, editing, debugging, and managing the project files.
Streamlit	Used to create the graphical user interface.
Pandas	Used to read and display structured tabular data required by the application.

The software components work together in a sequential manner:

CSV Data Sources → Python → Pandas → ETL & Data Validation → Analytical Data Tables → Power BI → Interactive Dashboards

This software environment provides all the necessary components for implementing the SalesSight 360 analytics workflow, from raw data processing to business intelligence reporting.

2.3 Hardware Requirements

The hardware requirements define the physical computing resources required to develop, execute, test, and visualize the **SalesSight 360 – Multi-Channel Revenue Analytics Platform**. Since the project involves data processing using Python and Pandas and interactive dashboard development using Power BI, a computer with adequate processing capability, memory, and storage is required. The project does not require specialized or high-end hardware because the datasets used are structured and relatively moderate in size. However, sufficient system resources are necessary to ensure smooth execution of Python scripts, data transformations, Power BI reports, and other development activities.

The software requirements used during the development of this project are listed below.

<i>Hardware</i>	<i>Specification</i>
Processor	Intel Core i7-13620H
RAM	16 GB
Storage	1 TB HDS
Operating System	Windows 11 (64-bit)
Display	Laptop Screen / Monitor

The above hardware configuration was sufficient for developing, testing, and executing the complete application without any noticeable performance issues.

2.4 Programming Language Used

Python

Python is the primary programming language used for the development of the SalesSight 360 – Multi-Channel Revenue Analytics Platform. Python was selected because of its simplicity, flexibility, extensive support for data analytics, and availability of powerful libraries for handling and processing structured datasets.

In this project, Python is mainly used for the ETL (Extract, Transform, Load) process, data preprocessing, data cleaning, data transformation, data validation, analytical calculations, and preparation of datasets for visualization. The raw business datasets are initially loaded into the Python environment. The data is then inspected to understand its structure and quality. Various preprocessing and transformation operations are performed to make the data consistent and suitable for analysis. Python is also used to create derived fields, perform aggregations, calculate business metrics, and prepare analytical tables. Major Uses of Python in SalesSight 360

Python is used for the following activities:

1. Data Extraction: Raw CSV datasets are loaded into the Python environment for further processing.
2. Data Inspection: Dataset structure, columns, data types, records, and data quality are examined before transformation.
3. Data Cleaning: Inconsistent data types, missing or invalid values, duplicate records, and other data-quality issues are handled as required.
4. Data Transformation: Raw datasets are transformed into structured and analysis-ready formats. Date fields are converted appropriately and additional fields such as Month are created for time-based analysis.

5. Data Integration: Related datasets are combined using common identifiers such as Customer_ID, Product_ID, and Order_ID.
6. Data Validation: Python-based validation checks are performed to identify invalid relationships between important datasets. For example, customer, product, and order identifiers are checked for consistency.
7. Analytical Calculations: Python is used to calculate important business metrics such as total sales, total profit, average order value, discount amount, net sales, conversion rate, cost per lead, and other KPIs.
8. Data Modelling Preparation: Processed data is organized into dimension and fact-style analytical tables such as DIM_CUSTOMERS, DIM_PRODUCTS, FACT_ORDERS, FACT_PAYMENTS, FACT_CRM, and FACT_MARKETING.
9. Power BI Data Preparation: The final processed datasets are prepared in a format suitable for importing into Microsoft Power BI for dashboard development and visualization.

Python Libraries Used

The major Python library used in the project is Pandas.

Library	Purpose
Pandas	Data loading, cleaning, transformation, merging, aggregation, analysis, and preparation of analytical datasets

Python was selected because it provides:

- Simple and readable syntax.
- Strong support for data analytics.
- Efficient handling of tabular datasets.
- Powerful data manipulation capabilities through Pandas.
- Easy implementation of ETL workflows.
 - Support for automation and reusable scripts.

Thus, Python acts as the primary data-processing and ETL layer of SalesSight 360, while Power BI acts as the business intelligence and visualization layer. Together, they provide an effective workflow for transforming raw business data into meaningful analytical insights.

2.5 Technologies and Libraries Used

The development of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** involves a combination of programming technologies, data-processing libraries, development tools, data-storage formats, and business intelligence technologies. Each technology is used for a specific stage of the project, starting from raw data processing and ending with interactive dashboard development.

2.5.1 Pandas

Pandas is the main data-analysis and data-manipulation library used in SalesSight 360. It provides powerful DataFrame structures for working with tabular datasets.

Pandas is used for:

- Reading and loading CSV files.
- Inspecting datasets and their structures.
- Cleaning and preprocessing data.
- Handling data types and date fields.
- Creating derived columns.
- Filtering and grouping records.
- Merging related datasets.
- Performing aggregations and calculations.
- Preparing fact and dimension tables.
- Generating analysis-ready datasets.

2.5.2 Microsoft Power BI

Microsoft Power BI is used as the business intelligence and data visualization technology of the project. The processed datasets are imported into Power BI to create interactive dashboards and reports. Power BI is used to display:

- Sales and profit KPIs.
- Monthly sales trends.
- Product performance.
- Customer insights.
- Marketing campaign performance.

- CRM pipeline metrics.
- Target-versus-actual performance.
- Interactive slicers and filters.

Power BI converts the processed analytical data into an easy-to-understand visual format that supports business decision-making.

2.5.3 Visual Studio Code

Visual Studio Code (VS Code) is used as the primary development environment for writing and executing Python programs. It provides an organized workspace for maintaining Python scripts, project files, datasets, and outputs. It is mainly used for developing and testing the ETL workflow and resolving programming or data-processing errors.

2.5.4 CSV

Comma-Separated Values (CSV) is used as the primary data-storage and exchange format for the source datasets. The project uses CSV files containing information related to customers, products, orders, payments, marketing, CRM, and sales targets. CSV files are lightweight, portable, and can be easily processed using Pandas and imported into Power BI.

2.5.5 ETL Technology

The project implements an **ETL (Extract, Transform, Load)** workflow using Python and Pandas.

- **Extract:** Data is collected from the source CSV files.
- **Transform:** Data is cleaned, standardized, integrated, and converted into analysis-ready structures.
- **Load:** The processed datasets are prepared for analytical use and Power BI visualization.

2.5.6 Data Modelling

A structured analytical data model is created to organize the processed information. The model includes dimension and fact-style tables such as:

- **DIM_CUSTOMERS**
- **DIM_PRODUCTS**
- **FACT_ORDERS**
- **FACT_PAYMENTS**
- **FACT_CRM**

- **FACT_MARKETING**

This structure helps maintain organized relationships between different business entities and supports consistent analytical reporting.

2.6 Development Tools Used

The development of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** requires different software tools for coding, data processing, testing, data management, and business intelligence reporting. These tools support the complete development lifecycle of the project, from writing the ETL scripts to creating the final interactive dashboards.

2.6.1 Visual Studio Code

Visual Studio Code (VS Code) is the primary development tool used for writing and executing the Python programs of the SalesSight 360 project.

It is used for:

- Writing Python and ETL scripts.
- Managing project files and folders.
- Executing Python programs.
- Testing data-processing operations.
- Identifying and correcting errors.
- Viewing program outputs.
- Maintaining reusable project code.

VS Code provides an organized development environment that makes it convenient to work with Python scripts and project datasets.

2.6.2 Python Development Environment

The Python development environment is used to execute the data-processing and ETL workflow. It provides the runtime environment required to run Python scripts and perform operations on the project datasets.

The environment is used for:

- Loading source datasets.

- Data preprocessing.
- Data transformation.
- Data integration.
- Data validation.
- KPI calculations.
- Generation of analytical datasets.

2.6.3 Microsoft Power BI Desktop

Microsoft Power BI Desktop is the primary reporting and visualization tool used in the project.

It is used to convert processed analytical data into interactive dashboards and reports. The tool provides different visualization options such as KPI cards, bar charts, line charts, tables, slicers, and other interactive elements. Power BI is used for developing dashboards related to:

- Executive business performance.
- Sales performance.
- Marketing and customer insights.
- CRM and pipeline analysis.
- Target-versus-actual performance.

2.6.4 File and Dataset Management

File management tools provided by the operating system are used to organize the project structure. Separate folders are maintained for raw datasets, cleaned/processed data, Python scripts, analytical outputs, Power BI files, and documentation. This organization helps maintain a clear separation between source data and processed data and makes the project easier to maintain.

2.6.5 Data Validation and Testing

Testing and validation are performed during the development process to ensure that the transformed data is accurate and suitable for analysis. The project includes checks such as:

- Checking dataset dimensions and columns.
- Verifying data types.
- Checking relationships between identifiers.

- Checking for invalid Customer_ID values.
- Checking for invalid Product_ID values.
- Checking for invalid Order_ID references.
- Validating calculated KPIs.
- Reviewing analytical outputs before dashboard development.

These checks help improve the reliability and consistency of the final analytical results.

2.6.6 Development Tools Summary

<u>Development Tool</u>	<u>Purpose</u>
Visual Studio Code	Python coding, execution, debugging and project management
Python Environment	Running ETL and analytical scripts
Microsoft Power BI Desktop	Dashboard and report development
File Management Tools	Organizing datasets, scripts and project files
Data Validation Process	Testing data quality and analytical outputs

2.6.7 Development Workflow

The development tools work together throughout the project according to the following workflow:

Source Dataset Management → VS Code → Python/Pandas ETL → Data Validation → Processed Data → Power BI Desktop → Dashboard Development → Final Business Insights

Thus, the selected development tools provide a complete environment for developing, testing, and presenting the SalesSight 360 analytics solution.

2.7 Why These Technologies Were Selected

The technologies used in the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** were selected based on their suitability for data processing, ETL implementation, analytical modelling, business intelligence, visualization, and ease of development. The combination of Python, Pandas, Visual Studio Code, CSV, and Microsoft Power BI provides a complete and practical environment for developing the project.

2.7.1 Python

Python was selected as the primary programming language because it is widely used for data analytics and provides a simple, readable, and flexible programming environment. It supports efficient handling of structured datasets and provides libraries that simplify data processing and analytical operations. For SalesSight 360, Python is particularly suitable for implementing the ETL workflow, performing data transformations, validating datasets, and calculating business metrics.

2.7.2 Pandas

Pandas was selected because the project involves processing multiple structured and tabular datasets. Its DataFrame-based structure makes it convenient to load, inspect, clean, filter, merge, group, and transform business data. Pandas also simplifies the preparation of analytical tables and calculation of KPIs, making it well suited for the data-processing requirements of SalesSight 360.

2.7.3 Microsoft Power BI

Power BI was selected because the project requires interactive business intelligence dashboards rather than only static analytical outputs. Power BI provides a wide range of visualization and reporting features that make complex business information easier to understand. It allows SalesSight 360 to present sales, profit, marketing, CRM, customer, and target-performance information through interactive charts, KPI cards, tables, slicers, and filters.

2.7.4 Visual Studio Code

Visual Studio Code was selected as the development environment because it provides a lightweight and organized platform for writing, executing, testing, and maintaining Python programs. It supports efficient development of the ETL scripts and makes it easier to manage project files, identify errors, and modify the data-processing workflow.

2.7.5 CSV

CSV was selected as the primary source-data format because it is simple, lightweight, portable, and widely supported by data-processing and visualization tools. The project's business datasets can be easily loaded into Pandas and subsequently imported into Power BI.

2.7.6 ETL Approach

An ETL approach was selected because the project contains multiple raw datasets that require cleaning, transformation, integration, and validation before analysis. The ETL process provides a systematic method for converting raw data into reliable, analysis-ready information.

2.7.7 Data Modelling

A dimensional and fact-style data modelling approach was selected to organize different business entities and transactions in a structured manner. Tables such as **DIM_CUSTOMERS**, **DIM_PRODUCTS**, **FACT_ORDERS**, **FACT_PAYMENTS**, **FACT_CRM**, and **FACT_MARKETING** provide a logical structure for analytical reporting and help maintain consistency across different business analyses.

2.7.8 Overall Reason for Technology Selection

The selected technology stack provides a balance between **ease of development, data-processing capability, analytical flexibility, visualization, and practical implementation**. Python and Pandas handle the data-processing and ETL layer, while Power BI handles the business intelligence and visualization layer. Visual Studio Code provides the development environment, and CSV files provide a convenient source-data format. Therefore, the selected technologies are appropriate for developing an end-to-end analytics platform capable of transforming raw multi-source business data into structured information, meaningful KPIs, interactive dashboards, and actionable business insights.

CHAPTER 3

ANALYSIS DOCUMENT

3.1 Introduction

The analysis phase is an important stage in the development of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform**. The primary purpose of this phase is to understand the available business data, identify the requirements of the analytical system, examine the relationships between different datasets, and determine the information required for effective business performance analysis. SalesSight 360 deals with multiple business datasets related to **customers, products, orders, payments, marketing, Customer Relationship Management (CRM), and sales targets**. Since these datasets represent different aspects of business operations, analysing their structure and relationships is necessary before performing data transformation and developing dashboards. During the analysis phase, the available datasets are examined with respect to their fields, data types, number of records, relationships, and analytical relevance. The analysis helps identify which attributes can be used as dimensions, which fields represent measurable business activities, and which relationships are required to combine the datasets effectively. The analysis also focuses on identifying the major business questions that the system should answer. These include questions such as how much revenue is generated, how profitable the business is, which products and customers perform better, how effective marketing campaigns are, what value exists in the CRM pipeline, and how actual sales compare with predefined targets. The major analytical areas identified for SalesSight 360 are:

- **Sales Analysis** – analysis of revenue, profit, orders, discounts, net sales, and sales trends.
- **Customer Analysis** – understanding customer-level sales contribution and performance.
- **Product Analysis** – identifying product-level sales and performance patterns.
- **Marketing Analysis** – analysing leads, conversions, campaign performance, and marketing expenditure.
- **CRM Analysis** – analysing deal value and expected revenue from the sales pipeline.
- **Target Analysis** – comparing actual sales with predefined sales targets and calculating performance variance.
- **KPI Analysis** – calculating important business performance indicators for management reporting.

The analysis phase therefore provides the foundation for the subsequent **ETL, data modelling, and Power BI dashboard development** stages. By understanding the data and defining the required analytical outputs before implementation, the project ensures that the final system is structured according to the actual requirements of the business analysis process. The overall analysis flow can be represented as:

3.2 Functional Requirements

Functional requirements define the specific functions and operations that the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** is expected to perform. These requirements describe how the system should process business data, generate analytical information, calculate performance indicators, and present the results through interactive dashboards. The major functional requirements of SalesSight 360 are as follows:

FR-01: Data Import

The system shall allow the required business datasets to be loaded from CSV files. The datasets include customer, product, order, payment, marketing, CRM, and sales target data.

FR-02: Data Inspection

The system shall inspect the imported datasets to identify their structure, columns, data types, record counts, and potential data-quality issues before further processing.

FR-03: Data Cleaning

The system shall clean and preprocess the source datasets by handling data inconsistencies, correcting data types, preparing required fields, and removing or addressing invalid records where necessary.

FR-04: Data Transformation

The system shall transform raw data into analysis-ready formats. This includes date conversion, creation of derived fields such as Month, calculation of required measures, and standardization of relevant data attributes.

FR-05: Data Integration

The system shall integrate related datasets using common identifiers such as **Customer_ID**, **Product_ID**, and **Order_ID** to establish meaningful relationships between business entities.

FR-06: Data Validation

The system shall perform validation checks to ensure the reliability and consistency of the processed data. The system shall verify relationships between customers, products, orders, and payments and identify invalid references.

FR-07: Analytical Data Model Creation

The system shall create structured analytical tables for reporting. These include:

- **DIM_CUSTOMERS**
- **DIM_PRODUCTS**
- **FACT_ORDERS**
- **FACT_PAYMENTS**
- **FACT_CRM**
- **FACT_MARKETING**

FR-08: Sales Analysis

The system shall calculate and provide sales-related metrics including:

- Total Sales
- Total Profit
- Total Orders
- Average Order Value
- Profit Margin
- Discount Amount
- Net Sales
- Monthly Sales

FR-09: Customer and Product Analysis

The system shall provide customer-level and product-level analysis to identify sales contribution, performance patterns, and high-performing customers or products.

FR-10: Marketing Analysis

The system shall analyse marketing performance using metrics such as:

- Total Leads
- Total Conversions
- Marketing Spend
- Conversion Rate

- Cost per Lead
- Cost per Conversion
- Campaign Performance

FR-11: CRM Analysis

The system shall provide analysis of CRM pipeline information, including:

- Total Deal Value
- Expected Revenue

This functionality shall help provide an overview of potential revenue represented in the CRM pipeline.

FR-12: Target versus Actual Analysis

The system shall compare actual sales with predefined sales targets and calculate:

- Total Sales Target
- Actual Sales
- Sales Variance
- Target Achievement Percentage

FR-13: KPI Generation

The system shall generate important business Key Performance Indicators (KPIs) from the processed datasets. These KPIs shall provide a summarized view of business performance.

FR-14: Dashboard Visualization

The system shall provide interactive Power BI dashboards for presenting the processed analytical information through KPI cards, charts, tables, trends, and other suitable visualizations.

FR-15: Interactive Filtering

The dashboards shall provide appropriate slicers and filters to allow users to explore the data based on available business dimensions and obtain more focused analytical views.

FR-16: Dashboard Navigation

The Power BI report shall provide navigation between relevant dashboard pages so that users can move between different analytical areas such as sales, marketing, customer insights, CRM, and performance analysis.

FR-17: Business Insight Generation

The system shall enable users to identify important trends, performance gaps, and patterns from the available business data and support data-driven business decision-making.

Summary of Functional Requirements

<u>Requirement ID</u>	<u>Functional Requirement</u>
FR-01	Data Import
FR-02	Data Inspection
FR-03	Data Cleaning
FR-04	Data Transformation
FR-05	Data Integration
FR-06	Data Validation
FR-07	Analytical Data Model
FR-08	Sales Analysis
FR-09	Customer & Product Analysis
FR-10	Marketing Analysis
FR-11	CRM Analysis
FR-12	Target vs Actual Analysis
FR-13	KPI Generation
FR-14	Dashboard Visualization
FR-15	Interactive Filtering
FR-16	Dashboard Navigation
FR-17	Business Insight Generation

These functional requirements define the core capabilities of SalesSight 360 and provide a clear foundation for the implementation of the ETL process, analytical data model, and Power BI dashboards

3.3 Data Flow Diagram (DFD Level 0)

Main Process: 0 – SalesSight 360 Multi-Channel Revenue Analytics Platform

External Entities:

- Source Data (CSV Files)
- Administrator / Analyst
- Business Users / Management
- Power BI

Data Store:

- D1 – Analytical Data Store (Fact & Dimension Tables)

Main Data Flows:

- Source Data → SalesSight 360: **Raw Data (CSV Files)**
- Administrator/Analyst → System: **Requests / Parameters**
- System → Analytical Data Store: **Processed & Validated Data**
- System → Power BI: **Processed Data for Visualization**
- Power BI → Business Users: **Interactive Dashboards & Insights**
- Business Users → System: **Filters, Selections & Drill-downs**
- System → Business Users: **Analytics Results, Dashboards & Reports**

The Level-0 Data Flow Diagram represents SalesSight 360 as a single integrated analytical process. Raw business data from CSV files is provided to the system, where it is processed, transformed, and validated. The resulting analytical data is stored in the analytical data store and supplied to Power BI for visualization. Business users and management can interact with the dashboards using filters and drill-downs and obtain analytical results and business insights.

3.4 Data Flow Diagram (DFD Level 1)

The **Level-1 Data Flow Diagram (DFD)** provides a detailed representation of the internal processes of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform**. Unlike Level 0, where the entire system is represented as a single process, Level 1 divides the system into major subprocesses involved in data processing and business intelligence. The major processes of the Level-1 DFD are:

Analysis Summary: The analysis phase of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** was carried out to understand the available business data, identify analytical requirements, define system functionality, and establish the relationships between different datasets. The analysis identified **seven major data areas**: customers, products, orders, payments, marketing, CRM, and sales targets. These datasets contain complementary information and therefore need to be integrated before meaningful business analysis can be performed. The major findings of the analysis are summarized below:

- Multiple business datasets need to be processed through a common analytical workflow.
- Raw datasets require **cleaning, preprocessing, transformation, and validation** before analysis.
- Common identifiers such as **Customer_ID, Product_ID, and Order_ID** are important for establishing relationships between datasets.
- The analytical model requires separate **dimension and fact-style tables** for organized reporting.
- Important KPIs need to be calculated to provide a summarized view of business performance.
- Sales analysis should cover revenue, profit, orders, discounts, net sales, and monthly trends.
- Marketing analysis should focus on leads, conversions, spending, conversion rate, and campaign performance.
- CRM analysis should provide information about deal value and expected revenue.
- Target analysis should compare actual sales with predefined targets and identify performance variance.
- Power BI dashboards are required to present the analytical results through interactive visualizations, filters, and KPI cards.
- Data validation is required to ensure that relationships between the datasets remain consistent and reliable.

The analysis therefore establishes the foundation for the subsequent **system design, ETL implementation, analytical data modelling, and Power BI dashboard development**. The final system is designed to transform raw multi-source business data into structured analytical information and meaningful business insights.

CHAPTER-4

SYSTEM IMPLEMENTATION

4.1 Introduction

The design phase of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** focuses on converting the requirements identified during the analysis phase into a structured technical and analytical design. The purpose of this phase is to define how different components of the system will interact, how business data will be organized, and how the processed information will be presented to users. SalesSight 360 is designed as an **end-to-end data analytics and business intelligence solution**. The design integrates the data-processing capabilities of Python and Pandas with the visualization and reporting capabilities of Microsoft Power BI. The system follows a structured flow in which raw business datasets are collected, processed, transformed, validated, modelled, and finally visualized through interactive dashboards. The design primarily focuses on the following components:

- **Source Data Design** – organization of customer, product, order, payment, marketing, CRM, and sales-target datasets.
- **ETL Design** – extraction, cleaning, transformation, integration, and validation of raw data.
- **Data Model Design** – organization of processed data into dimension and fact-style tables.
- **KPI Design** – definition and calculation of important business performance indicators.
- **Dashboard Design** – development of interactive Power BI pages for different business analysis areas.
- **User Interaction Design** – use of filters, slicers, navigation, and interactive visualizations.
- **Reporting Design** – presentation of analytical results in a clear and understandable format.

4.2 Overall System Architecture

The **Overall System Architecture** of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** describes how the different components of the system work together to convert raw business data into meaningful business intelligence. The architecture follows a layered approach in which data moves through the stages of **data collection, processing, validation, modelling, analysis, visualization, and reporting**.

4.3 Modular Design

The **Modular Design** of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** divides the complete system into a set of independent but interconnected modules. Each module performs a specific function of the analytics workflow. This approach makes the system easier to develop, test, maintain, and extend.

4.4 Module Description

The SalesSight 360 – Multi-Channel Revenue Analytics Platform is developed using a modular Python-based architecture. The complete project is divided into multiple modules, where each module performs a specific data processing, analytical, or reporting task. This modular approach makes the system organized, maintainable, reusable, and easier to test.

4.4.1 ETL Module – etl.py

The etl.py module performs the main Extract, Transform, and Load (ETL) operations of the project. It loads the raw business datasets, performs preprocessing and cleaning, converts required data types, creates derived fields, and prepares the datasets for further analysis. The module also performs data validation to ensure that relationships between important identifiers such as Customer_ID, Product_ID, and Order_ID are correct. The major output of this module includes cleaned datasets such as customers_cleaned, orders_cleaned, products_cleaned, payments_cleaned, marketing_cleaned, crm_cleaned, and target_cleaned.

4.4.2 Data Modelling Module – data_modal.py

The data_modal.py module is responsible for creating the structured analytical data model from the cleaned datasets. It organizes the processed information into fact and dimension tables required for business intelligence analysis.

4.4.3 Customer Analysis Module – customer_analysis.py

The customer_analysis.py module performs customer-related analysis using the processed customer and order information. It helps identify customer contribution to sales, orders, and profit.

The module generates analytical outputs such as:

- Customer performance analysis
- Top customers by sales
- Top customers by profit
- Top customers by number of orders

4.4.4 Product Analysis Module – product_analysis.py

The product_analysis.py module analyses product-level performance. It evaluates products based on sales, quantity sold, and profit contribution.

4.4.5 Sales Trend Analysis Module – sales_trend_analysis.py

The sales_trend_analysis.py module performs time-based sales analysis. It uses the processed order-date information to identify monthly sales trends and variations. The module generates the sales_trend_analysis output, which can be used to visualize sales performance over time and identify periods of higher or lower sales.

4.4.6 Marketing Analysis Module – marketing_analysis.py

The marketing_analysis.py module performs analysis of marketing campaigns and channels. It evaluates leads, conversions, marketing expenditure, conversion rates, and campaign effectiveness.

4.4.7 CRM Analysis Module – crm_analysis.py

The crm_analysis.py module analyses customer relationship management data. It evaluates CRM deals, deal values, expected revenue, and different stages of the sales pipeline. This module helps in understanding the sales pipeline and potential future revenue.

4.4.8 Target Analysis Module – target_analysis.py

The target_analysis.py module compares actual sales performance with predefined sales targets. It calculates important performance measures such as sales variance and target achievement percentage. The module uses the fact_target and sales-related data to identify performance gaps between actual and expected sales.

4.4.9 KPI Analysis Module – kpi_analysis.py

The kpi_analysis.py module calculates the major Key Performance Indicators (KPIs) of the SalesSight 360 project. The KPI outputs provide a summarized view of overall business performance.

4.4.10 Analytical Output Module

The Python modules generate several processed Excel/CSV-style analytical outputs. These outputs act as intermediate analytical datasets and are used for further analysis and Power BI reporting.

4.4.11 Power BI Reporting Module

The processed outputs generated by the Python modules are used in Microsoft Power BI to develop interactive dashboards. The dashboards provide a consolidated view of sales, customers, products, marketing, CRM, and target performance.

4.5 Database Schema

The **database scheme of SalesSight 360 – Multi-Channel Revenue Analytics Platform** defines the logical organization of the processed business data and the relationships between different analytical tables. Since the project uses a **Python-based ETL process with CSV/Excel-based datasets and Power BI for reporting**, the database scheme is implemented as an **analytical data model** rather than a traditional transactional database. The scheme is designed to integrate data from different business areas such as customers, products, orders, payments, marketing, CRM, and sales targets.

Main Tables

The analytical database scheme consists of the following major tables:

<u>Table</u>	<u>Type</u>	<u>Purpose</u>
DIM_CUSTOMERS	Dimension	Stores customer-related information
DIM_PRODUCTS	Dimension	Stores product-related information
FACT_ORDERS	Fact	Stores order and sales transaction data
FACT_PAYMENTS	Fact	Stores payment-related transaction data
FACT_CRM	Fact	Stores CRM deal and pipeline information
FACT_MARKETING	Fact	Stores marketing campaign and performance data
FACT_TARGET	Fact	Stores sales target information

4.6 Data Integrity Constraints

Data integrity constraints ensure that the data used in **SalesSight 360** remains accurate, consistent, and reliable.

The major constraints are:

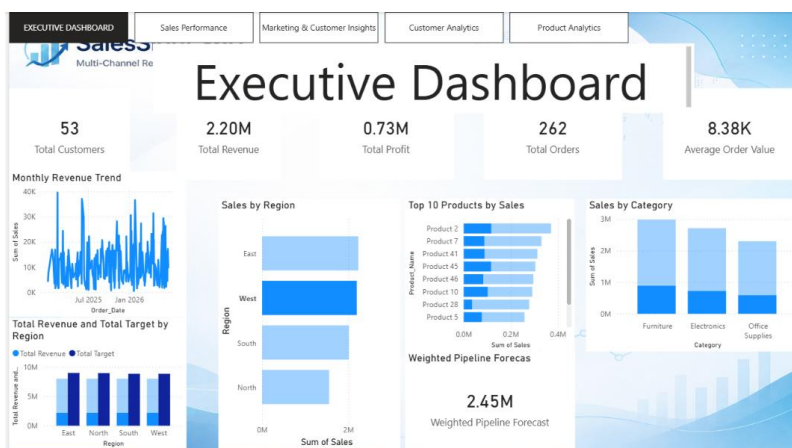
- **Entity Integrity:** Unique IDs such as Customer_ID, Product_ID, and Order_ID identify records correctly.
- **Referential Integrity:** Related IDs must exist in their corresponding tables.
- **Domain Integrity:** Data must follow valid formats, ranges, and data types.
- **Uniqueness:** Important identifiers should not contain unwanted duplicate records.

4.7 User Interface (UI) Design

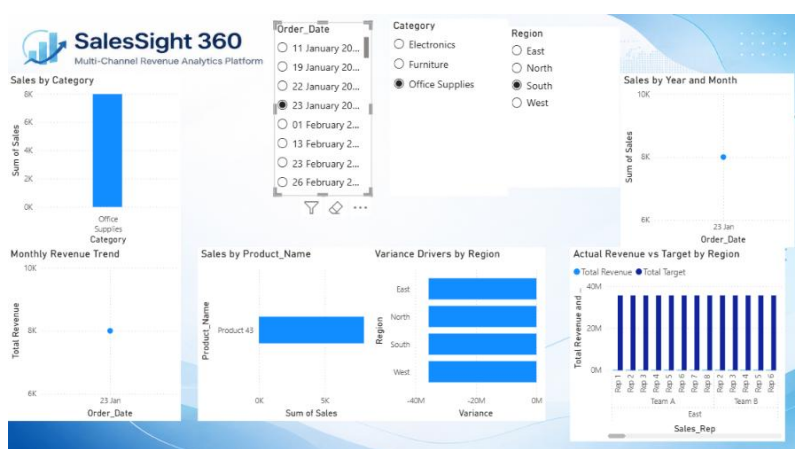
The **User Interface (UI) Design** of the SalesSight 360 project is implemented using **Microsoft Power BI**. The main objective of the UI design is to present complex business data in a simple, interactive, and easy-to-understand format. The dashboard interface includes:

- **KPI Cards** – Display important metrics such as Sales, Profit, Orders, and Target Achievement.
- **Charts and Graphs** – Represent sales trends, product performance, marketing performance, and CRM analysis visually.
- **Tables** – Provide detailed analytical information.
- **Slicers and Filters** – Allow users to filter data according to available dimensions.
- **Navigation** – Enables users to move between different dashboard pages.
- **Interactive Visuals** – Allow users to explore business data and identify trends and performance gaps.

4.7.1 Dashboard Page



Sales Performance



Marketing And Customer Insight



4.7.2 Customer Analytics



4.6.5 Product Analytics



CHAPTER 5

PROGRAM CODE (SYSTEM IMPLEMENTATION)

5.1 Introduction

The implementation phase involves converting the design of the SalesSight 360 – Multi-Channel Revenue Analytics Platform into an operational data analytics solution. The system is implemented primarily using Python, Pandas, and Microsoft Power BI. Python modules are used to perform the major data-processing and analytical operations. The implementation starts with loading the raw business datasets and continues through data cleaning, transformation, integration, validation, analytical modelling, KPI calculation, and generation of analytical outputs.

5.2 Main Module (etl.py)

The **main module of the SalesSight 360 project is etl.py**. It acts as the central processing module and coordinates the major data preparation activities required before analysis and visualization.

Source Code:

```
import pandas as pd

import os

DATA_FOLDER = "data"

customers = pd.read_csv(os.path.join(DATA_FOLDER, "customers.csv"))

orders = pd.read_csv(os.path.join(DATA_FOLDER, "orders.csv"))

products = pd.read_csv(os.path.join(DATA_FOLDER, "products.csv"))

payments = pd.read_csv(os.path.join(DATA_FOLDER, "payments.csv"))

marketing = pd.read_csv(os.path.join(DATA_FOLDER, "marketing.csv"))

crm = pd.read_csv(os.path.join(DATA_FOLDER, "crm.csv"))

target = pd.read_csv(os.path.join(DATA_FOLDER, "target.csv"))

print("All data files loaded successfully!")

print("\nCustomers:", customers.shape)

print("Orders:", orders.shape)
```

```
print("Products:", products.shape)

print("Payments:", payments.shape)

print("Marketing:", marketing.shape)

print("CRM:", crm.shape)

print("Target:", target.shape)

customers = customers.drop_duplicates()

orders = orders.drop_duplicates()

products = products.drop_duplicates()

payments = payments.drop_duplicates()

marketing = marketing.drop_duplicates()

crm = crm.drop_duplicates()

target = target.drop_duplicates()

customers.columns = customers.columns.str.strip()

orders.columns = orders.columns.str.strip()

products.columns = products.columns.str.strip()

payments.columns = payments.columns.str.strip()

marketing.columns = marketing.columns.str.strip()

crm.columns = crm.columns.str.strip()

target.columns = target.columns.str.strip()

orders["Order_Date"] = pd.to_datetime(
    orders["Order_Date"],
    errors="coerce")

crm["Lead_Date"] = pd.to_datetime(
    crm["Lead_Date"],
    errors="coerce")
```

```

crm["Close_Date"] = pd.to_datetime(

    crm["Close_Date"],

    errors="coerce")

for df in [orders, payments, marketing, crm, target]:

    numeric_columns = df.select_dtypes(include="number").columns

    df[numeric_columns] = df[numeric_columns].fillna(0)

for df in [

    customers,

    orders,

    products,

    payments,

    marketing,

    crm,

    target]:

    text_columns = df.select_dtypes(include="object").columns

    for col in text_columns:

        df[col] = df[col].str.strip()

print("\nData cleaning completed successfully!")

OUTPUT_FOLDER = "."

customers.to_csv("customers_cleaned.csv", index=False)

orders.to_csv("orders_cleaned.csv", index=False)

products.to_csv("products_cleaned.csv", index=False)

payments.to_csv("payments_cleaned.csv", index=False)

marketing.to_csv("marketing_cleaned.csv", index=False)

crm.to_csv("crm_cleaned.csv", index=False)

```

```

target.to_csv("target_cleaned.csv", index=False)

print("\nCleaned datasets saved successfully!")

print("ETL PROCESS COMPLETED SUCCESSFULLY! 🎉")

print("\n===== CLEANED DATA VALIDATION =====")

datasets = {

    "Customers": customers,

    "Orders": orders,

    "Products": products,

    "Payments": payments,

    "Marketing": marketing,

    "CRM": crm,

    "Target": target}

for name, df in datasets.items():

    print(f"\n{name}")

    print("Rows:", df.shape[0])

    print("Columns:", df.shape[1])

    print("Missing Values:")

    print(df.isnull().sum())

    print("-" * 50)

print("\nDATA VALIDATION COMPLETED SUCCESSFULLY! ✅")

```

5.3 Data Module(data_modal.py)

The data_modal.py module is responsible for creating the structured analytical data model of the SalesSight 360 project. It takes the cleaned and validated datasets generated during the ETL process and organizes them into fact and dimension tables for further analysis and Power BI reporting.

Source code:

```
import pandas as pd
```

```
# LOAD CLEANED DATASETS
```

```
customers = pd.read_csv("customers_cleaned.csv")
```

```
orders = pd.read_csv("orders_cleaned.csv")
```

```
products = pd.read_csv("products_cleaned.csv")
```

```
payments = pd.read_csv("payments_cleaned.csv")
```

```
marketing = pd.read_csv("marketing_cleaned.csv")
```

```
crm = pd.read_csv("crm_cleaned.csv")
```

```
target = pd.read_csv("target_cleaned.csv")
```

```
print("All cleaned datasets loaded successfully! ")
```

```
print("\n===== DATA MODEL VALIDATION =====")
```

```
print("\n1. CUSTOMER - ORDER RELATIONSHIP")
```

```
missing_customers = orders[
```

```
    ~orders["Customer_ID"].isin(customers["Customer_ID"])]
```

```
print("Orders with invalid Customer_ID:", len(missing_customers))
```

```
print("\n2. PRODUCT - ORDER RELATIONSHIP")
```

```
missing_products = orders[
```

```
    ~orders["Product_ID"].isin(products["Product_ID"])
```

```
]
```

```
print("Orders with invalid Product_ID:", len(missing_products))
```

```
print("\n3. ORDER - PAYMENT RELATIONSHIP")
```

```
missing_orders = payments[
```

```
    ~payments["Order_ID"].isin(orders["Order_ID"])]
```

```
print("Payments with invalid Order_ID:", len(missing_orders))
```

```
print("\n===== DATA MODEL SUMMARY =====")
```

```
print("Customers:", customers.shape)
```

```

print("Orders:", orders.shape)

print("Products:", products.shape)

print("Payments:", payments.shape)

print("Marketing:", marketing.shape)

print("CRM:", crm.shape)

print("Target:", target.shape)

if (
    len(missing_customers) == 0
    and len(missing_products) == 0
    and len(missing_orders) == 0):

    print("\n✅ ALL CORE DATA RELATIONSHIPS ARE VALID!")

    print("DATA MODEL VALIDATION COMPLETED SUCCESSFULLY! 🎉")

else:

    print("\n⚠️ SOME DATA RELATIONSHIPS NEED ATTENTION.")

print("\n===== CREATING ANALYTICS DATA MODEL =====")

dim_customers = customers.copy()

dim_products = products.copy()

fact_orders = orders.copy()

fact_payments = payments.copy()

fact_crm = crm.copy()

fact_marketing = marketing.copy()

fact_target = target.copy()

fact_orders["Profit_Margin"] = (
    fact_orders["Profit"] / fact_orders["Sales"]
) * 100

```

```

fact_orders["Discount_Amount"] = (
    fact_orders["Sales"] * fact_orders["Discount"])

fact_orders["Net_Sales"] = (
    fact_orders["Sales"] - fact_orders["Discount_Amount"]
)

fact_crm["Expected_Revenue"] = (
    fact_crm["Deal_Value"] * fact_crm["Probability"]
)

fact_marketing["Conversion_Rate"] = (
    fact_marketing["Conversions"] / fact_marketing["Leads"]
) * 100

fact_marketing["Cost_Per_Lead"] = (
    fact_marketing["Spend"] / fact_marketing["Leads"])

fact_marketing["Cost_Per_Conversion"] = (
    fact_marketing["Spend"] / fact_marketing["Conversions"])

dim_customers.to_csv(
    "dim_customers.csv",
    index=False)

dim_products.to_csv(
    "dim_products.csv",
    index=False)

fact_orders.to_csv(
    "fact_orders.csv",
    index=False)

fact_payments.to_csv(

```

```
"fact_payments.csv",

index=False)

fact_crm.to_csv(

    "fact_crm.csv",

    index=False)

fact_marketing.to_csv(

    "fact_marketing.csv",

    index=False)

fact_target.to_csv(

    "fact_target.csv",

    index=False)

print("\n===== ANALYTICS DATA MODEL =====")

print("Dimension Tables:")

print("DIM_CUSTOMERS:", dim_customers.shape)

print("DIM_PRODUCTS:", dim_products.shape)

print("\nFact Tables:")

print("FACT_ORDERS:", fact_orders.shape)

print("FACT_PAYMENTS:", fact_payments.shape)

print("FACT_CRM:", fact_crm.shape)

print("FACT_MARKETING:", fact_marketing.shape)

print("FACT_TARGET:", fact_target.shape)

print("\n✅ ANALYTICS DATA MODEL CREATED SUCCESSFULLY!")

print("✅ KPI COLUMNS CREATED SUCCESSFULLY!")

print("✅ ANALYTICS TABLES SAVED SUCCESSFULLY! 🎉")
```

5.4 Customer Analysis(customer_analysis.py)

The customer_analysis.py module is responsible for performing detailed customer-level analysis in the SalesSight 360 project. It uses the processed customer and order data to evaluate customer contribution and performance.

Source code:

```
import pandas as pd

fact_orders = pd.read_csv("fact_orders.csv")

dim_customers = pd.read_csv("dim_customers.csv")

print("Analytics tables loaded successfully! )

print("\n===== CUSTOMER PERFORMANCE ANALYSIS =====")

customer_analysis = (

    fact_orders

    .groupby("Customer_ID", as_index=False)

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Total_Orders=("Order_ID", "nunique"),

        Total_Quantity=("Quantity", "sum")

    )

)

customer_analysis["Profit_Margin_%"] = (

    customer_analysis["Total_Profit"]

    / customer_analysis["Total_Sales"]

) * 100

# ADD CUSTOMER INFORMATION

customer_columns = [
```

```

col for col in dim_customers.columns

if col != "Customer_ID"

]

customer_analysis = pd.merge(

    customer_analysis,

    dim_customers,

    on="Customer_ID",

    how="left"

)

print("\n===== TOP 10 CUSTOMERS BY SALES =====")

top_sales_customers = (

    customer_analysis

    .sort_values(

        by="Total_Sales",

        ascending=False

    )

    .head(10)

)

print(top_sales_customers)

# TOP 10 CUSTOMERS BY PROFIT

print("\n===== TOP 10 CUSTOMERS BY PROFIT =====")

top_profit_customers = (

    customer_analysis

    .sort_values(

        by="Total_Profit",

```

```

        ascending=False

    )

    .head(10)

)

print(top_profit_customers)

print("\n===== TOP 10 CUSTOMERS BY ORDER COUNT =====")

top_order_customers = (

    customer_analysis

    .sort_values(

        by="Total_Orders",

        ascending=False

    )

    .head(10)

)

print(top_order_customers)

# LOWEST PERFORMING CUSTOMERS

print("\n===== LOWEST PERFORMING CUSTOMERS =====")

lowest_sales_customers = (

    customer_analysis

    .sort_values(

        by="Total_Sales",

        ascending=True

    )

    .head(10)

)

```

```

print(lowest_sales_customers)

customer_analysis.to_csv(
    "customer_performance_analysis.csv",
    index=False
)

top_sales_customers.to_csv(
    "top_10_customers_by_sales.csv",
    index=False
)

top_profit_customers.to_csv(
    "top_10_customers_by_profit.csv",
    index=False
)

top_order_customers.to_csv(
    "top_10_customers_by_orders.csv",
    index=False
)

best_sales_customer = (
    customer_analysis
    .sort_values(
        by="Total_Sales",
        ascending=False
    )
    .iloc[0]
)

```

```

print("\n===== BEST CUSTOMER BY SALES =====")

print(
    "Customer ID:",
    best_sales_customer["Customer_ID"]
)print(
    "Total Sales:",
    best_sales_customer["Total_Sales"]
)
print(
    "Total Profit:",
    best_sales_customer["Total_Profit"]
)print(
    "Total Orders:",
    best_sales_customer["Total_Orders"]
)
# BEST CUSTOMER BY PROFIT
best_profit_customer = (
    customer_analysis
    .sort_values(
        by="Total_Profit",
        ascending=False
    )
    .iloc[0]
)
print("\n===== BEST CUSTOMER BY PROFIT =====")

```

```

print(
    "Customer ID:",
    best_profit_customer["Customer_ID"]
)

print(
    "Total Profit:",
    best_profit_customer["Total_Profit"]
)

print(
    "Total Sales:",
    best_profit_customer["Total_Sales"]
)

most_frequent_customer = (
    customer_analysis
    .sort_values(
        by="Total_Orders",
        ascending=False
    )
    .iloc[0]
)

print("\n===== MOST FREQUENT CUSTOMER =====")

print(
    "Customer ID:",
    most_frequent_customer["Customer_ID"]
)

```

```

print(
    "Total Orders:",
    most_frequent_customer["Total_Orders"]
)

print(
    "Total Sales:",
    most_frequent_customer["Total_Sales"]
)

print("\n=====")

print("CUSTOMER PERFORMANCE ANALYSIS COMPLETED!")

print("=====")

print("✅ Customer-wise Sales Analysis Completed!")

print("✅ Customer-wise Profit Analysis Completed!")

print("✅ Customer Order Frequency Analysis Completed!")

print("✅ Top Customers Identified!")

print("✅ Low-performing Customers Identified!")

print("✅ Analysis Files Saved Successfully!")

print("\n🎉 CUSTOMER ANALYSIS COMPLETED SUCCESSFULLY!")

```

5.5 Product Performance Analysis(product_analysis.py)

The Product Performance Analysis module is responsible for analysing the performance of different products in the SalesSight 360 project. It uses the processed product and order data to evaluate product-wise sales, quantity sold, and profit contribution. The module helps identify the top-performing products and compare products based on their contribution to overall business performance.

Source code:

```
import pandas as pd
```

```

fact_orders = pd.read_csv("fact_orders.csv")

dim_products = pd.read_csv("dim_products.csv")

print("Analytics tables loaded successfully! )

# PRODUCT PERFORMANCE ANALYSIS

print("\n===== PRODUCT PERFORMANCE ANALYSIS =====")

product_analysis = (

    fact_orders

    .groupby("Product_ID", as_index=False)

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Total_Quantity=("Quantity", "sum"),

        Total_Orders=("Order_ID", "nunique")

    )

)

product_analysis["Profit_Margin_%"] = (

    product_analysis["Total_Profit"]

    / product_analysis["Total_Sales"]

) * 100

product_columns = [

    col for col in dim_products.columns

    if col != "Product_ID"

]

product_analysis = pd.merge(

    product_analysis,

```

```

dim_products,

on="Product_ID",

how="left"

)

product_analysis = product_analysis.sort_values(

    by="Total_Sales",

    ascending=False

)

print("\n===== TOP 10 PRODUCTS BY SALES =====")

top_sales_products = product_analysis.head(10)

print(top_sales_products)

print("\n===== TOP 10 PRODUCTS BY PROFIT =====")

top_profit_products = (

    product_analysis

    .sort_values(

        by="Total_Profit",

        ascending=False

    )

    .head(10)

)

print(top_profit_products)

print("\n===== TOP 10 PRODUCTS BY QUANTITY =====")

top_quantity_products = (

    product_analysis

    .sort_values(

```

```

        by="Total_Quantity",

        ascending=False

    )

    .head(10)

)

print(top_quantity_products)

print("\n===== LOWEST PERFORMING PRODUCTS =====")

lowest_sales_products = (

    product_analysis

    .sort_values(

        by="Total_Sales",

        ascending=True

    )

    .head(10)

)

product_analysis.to_csv(

    "product_performance_analysis.csv",

    index=False

)

top_sales_products.to_csv(

    "top_10_products_by_sales.csv",

    index=False

)

top_profit_products.to_csv(

    "top_10_products_by_profit.csv",

```

```

    index=False

)

top_quantity_products.to_csv(

    "top_10_products_by_quantity.csv",

    index=False

)

best_sales_product = product_analysis.iloc[0]

print("\n===== BEST PRODUCT BY SALES =====")

print(

    "Product ID:",

    best_sales_product["Product_ID"]

)

print(

    "Total Sales:",

    best_sales_product["Total_Sales"]

)

print(

    "Total Profit:",

    best_sales_product["Total_Profit"]

)

best_profit_product = (

    product_analysis

    .sort_values(

        by="Total_Profit",

        ascending=False

```

```

    )

    .iloc[0]

)

print("\n===== BEST PRODUCT BY PROFIT =====")

print(

    "Product ID:",

    best_profit_product["Product_ID"]

)

print(

    "Total Profit:",

    best_profit_product["Total_Profit"]

)

print(

    "Total Sales:",

    best_profit_product["Total_Sales"]

)

print("\n=====")

print("PRODUCT PERFORMANCE ANALYSIS COMPLETED!")

print("=====")

print("✅ Product-wise Sales Analysis Completed!")

print("✅ Product-wise Profit Analysis Completed!")

print("✅ Product-wise Quantity Analysis Completed!")

print("✅ Product-wise Profit Margin Calculated!")

print("✅ Top Products Identified!")

```

```
print("✅ Low-performing Products Identified!")
```

```
print("✅ Analysis Files Saved Successfully!")
```

```
print("\n🎉 PRODUCT ANALYSIS COMPLETED SUCCESSFULLY!")
```

5.6 Sales trend analysis(sales_analysis.py)

The **Sales Trend Analysis** module is responsible for analysing the changes in sales over time in the SalesSight 360 project. It uses the processed order data and the Order_Date field to generate **monthly sales trends**.

The module helps identify periods of higher and lower sales and provides a clear view of overall sales movement over time.

Source code:

```
import pandas as pd

fact_orders = pd.read_csv("fact_orders.csv")

print("Fact Orders table loaded successfully! ✅")

print("\n===== ORDERS TABLE COLUMNS =====")

fact_orders["Order_Date"] = pd.to_datetime(

    fact_orders["Order_Date"]

)

print("\nOrder_Date converted successfully! ✅")

fact_orders["Month"] = (

    fact_orders["Order_Date"]

    .dt.to_period("M")

    .astype(str)

)

print("\n===== MONTHLY SALES ANALYSIS =====")

monthly_sales = (

    fact_orders
```

```

.groupby("Month", as_index=False)["Sales"]

.sum()

)

monthly_sales = monthly_sales.rename(

    columns={

        "Sales": "Total_Sales"

    }

)

print(monthly_sales)

print("\n===== MONTHLY PROFIT ANALYSIS =====")

monthly_profit = (

    fact_orders

    .groupby("Month", as_index=False)["Profit"]

    .sum()

)

monthly_profit = monthly_profit.rename(

    columns={

        "Profit": "Total_Profit"

    }

)

print(monthly_profit)

print("\n===== MONTHLY ORDER ANALYSIS =====")

monthly_orders = (

    fact_orders

    .groupby("Month", as_index=False)["Order_ID"]

```

```

        .nunique()
    )
    monthly_orders = monthly_orders.rename(
        columns={
            "Order_ID": "Total_Orders"
        }
    )
    print(monthly_orders)
    print("\n===== COMBINED MONTHLY ANALYSIS =====")
    monthly_analysis = pd.merge(
        monthly_sales,
        monthly_profit,
        on="Month"
    )
    monthly_analysis = pd.merge(
        monthly_analysis,
        monthly_orders,
        on="Month"
    )
    print(monthly_analysis)
    monthly_analysis["Sales_Growth_%"] = (
        monthly_analysis["Total_Sales"]
        .pct_change()
        * 100
    )

```

```

print("\n===== MONTHLY GROWTH ANALYSIS =====")

print(monthly_analysis)

monthly_analysis.to_csv(

    "sales_trend_analysis.csv",

    index=False

)

print("\n✅ SALES TREND ANALYSIS SAVED SUCCESSFULLY!")

best_sales_month = monthly_analysis.loc[

    monthly_analysis["Total_Sales"].idxmax()

]

print("\n===== BEST SALES MONTH =====")

print(

    "Best Sales Month:",

    best_sales_month["Month"]

)

print(

    "Highest Sales:",

    best_sales_month["Total_Sales"]

)

best_profit_month = monthly_analysis.loc[

    monthly_analysis["Total_Profit"].idxmax()

]

print("\n===== BEST PROFIT MONTH =====")

print(

    "Best Profit Month:",

```

```

    best_profit_month["Month"]

)

print(

    "Highest Profit:",

    best_profit_month["Total_Profit"]

)

print("\n=====")

print("SALES TREND ANALYSIS COMPLETED!")

print("=====")

print("✅ Monthly Sales Analysis Completed!")

print("✅ Monthly Profit Analysis Completed!")

print("✅ Monthly Order Analysis Completed!")

print("✅ Monthly Growth Analysis Completed!")

print("✅ Best Sales Month Identified!")

print("✅ Best Profit Month Identified!")

print("\n🎉 SALES TREND ANALYSIS COMPLETED SUCCESSFULLY!")

```

5.7 Target Versus Actual Analysis(target_analysis.py)

The Target Versus Actual Analysis module compares the actual sales achieved by the business with the predefined sales targets. It helps identify performance gaps and measure the level of target achievement. The module uses target_analysis.py and the fact_target data to calculate important measures such as Sales Target, Actual Sales, Sales Variance, and Target Achievement Percentage.

Source code:

```

import pandas as pd

fact_orders = pd.read_csv("fact_orders.csv")

fact_target = pd.read_csv("fact_target.csv")

```

```

print("Analytics tables loaded successfully!  ")

print("\n===== ORDERS TABLE COLUMNS =====")

print(fact_orders.columns.tolist())

print("\n===== TARGET TABLE COLUMNS =====")

print(fact_target.columns.tolist())

print("\n===== ACTUAL SALES SUMMARY =====")

total_actual_sales = fact_orders["Sales"].sum()

print("Total Actual Sales:", total_actual_sales)

print("\n===== SALES TARGET SUMMARY =====")

total_sales_target = fact_target["Sales_Target"].sum()

print("Total Sales Target:", total_sales_target)

print("\n===== TARGET VS ACTUAL =====")

sales_variance = (

    total_actual_sales - total_sales_target

)

target_achievement = (

    total_actual_sales / total_sales_target

) * 100

print("Actual Sales:", total_actual_sales)

print("Sales Target:", total_sales_target)

print("Sales Variance:", sales_variance)

print("Target Achievement %:", target_achievement)

print("\n===== REGION-WISE TARGET ANALYSIS =====")

if "Region" in fact_orders.columns and "Region" in fact_target.columns:

```

```

region_actual_sales = (
    fact_orders
    .groupby("Region", as_index=False)["Sales"]
    .sum()
)

region_target_sales = (
    fact_target
    .groupby("Region", as_index=False)["Sales_Target"]
    .sum()
)

region_analysis = pd.merge(
    region_actual_sales,
    region_target_sales,
    on="Region",
    how="outer"
)

region_analysis["Sales"] = (
    region_analysis["Sales"].fillna(0)
)

region_analysis["Sales_Target"] = (
    region_analysis["Sales_Target"].fillna(0)
)

region_analysis["Sales_Variance"] = (
    region_analysis["Sales"]
    - region_analysis["Sales_Target"]
)

```

```

)

region_analysis["Achievement_%"] = (

    region_analysis["Sales"]

    / region_analysis["Sales_Target"]

) * 100

print(region_analysis)

region_analysis.to_csv(

    "region_target_analysis.csv",

    index=False

)

print("\n✅ Region analysis saved successfully!")

else:

    print(

        "⚠️ Region column is not available in both tables."

    )

print("\n===== TEAM-WISE TARGET ANALYSIS =====")

if "Team" in fact_orders.columns and "Team" in fact_target.columns:

    team_actual_sales = (

        fact_orders

        .groupby("Team", as_index=False)["Sales"]

        .sum()

    )

    team_target_sales = (

        fact_target

        .groupby("Team", as_index=False)["Sales_Target"]

```

```

        .sum()

    )

team_analysis = pd.merge(

    team_actual_sales,

    team_target_sales,

    on="Team",

    how="outer"

)

team_analysis["Sales"] = (

    team_analysis["Sales"].fillna(0)

)

team_analysis["Sales_Target"] = (

    team_analysis["Sales_Target"].fillna(0)

)

team_analysis["Sales_Variance"] = (

    team_analysis["Sales"]

    - team_analysis["Sales_Target"]

)

team_analysis["Achievement_%"] = (

    team_analysis["Sales"]

    / team_analysis["Sales_Target"]

) * 100

print(team_analysis)

team_analysis.to_csv(

    "team_target_analysis.csv",

```

```

        index=False

    )

    print("\n✅ Team analysis saved successfully!")

else:

    print(

        "⚠️ Team column is not available in both tables."

    )

print("\n===== MONTH-WISE TARGET ANALYSIS =====")

if "Month" in fact_orders.columns and "Month" in fact_target.columns:

    monthly_actual_sales = (

        fact_orders

        .groupby("Month", as_index=False)["Sales"]

        .sum()

    )

    monthly_target_sales = (

        fact_target

        .groupby("Month", as_index=False)["Sales_Target"]

        .sum()

    )

    monthly_analysis = pd.merge(

        monthly_actual_sales,

        monthly_target_sales,

        on="Month",

        how="outer"

    )

```

```

monthly_analysis["Sales"] = (
    monthly_analysis["Sales"].fillna(0)
)
monthly_analysis["Sales_Target"] = (
    monthly_analysis["Sales_Target"].fillna(0)
)
monthly_analysis["Sales_Variance"] = (
    monthly_analysis["Sales"]
    - monthly_analysis["Sales_Target"]
)
monthly_analysis["Achievement_%"] = (
    monthly_analysis["Sales"]
    / monthly_analysis["Sales_Target"]
) * 100
print(monthly_analysis)
monthly_analysis.to_csv(
    "monthly_target_analysis.csv",
    index=False
)
print("\n✅ Monthly analysis saved successfully!")
else:
    print(
        "⚠️ Month column is not available in both tables."
    )
print("\n=====")

```

```
print("TARGET VS ACTUAL ANALYSIS COMPLETED!")

print("=====")

print("\n✅ Overall Target vs Actual calculated!")

print("\n🎉 TARGET ANALYSIS PROCESS COMPLETED SUCCESSFULLY!")
```

5.8 Marketing and campaign analysis (marketing_analysis.py)

The Marketing and Campaign Analysis module is responsible for analysing marketing activities and measuring the performance of different campaigns and channels. It is implemented through the marketing_analysis.py module using the processed marketing data. The module analyses leads, conversions, marketing spend, conversion rate, cost per lead, and cost per conversion to evaluate marketing effectiveness.

Source code:

```
import pandas as pd

fact_marketing = pd.read_csv("fact_marketing.csv")

print("Marketing analytics table loaded successfully! ✅")

print("\n===== MARKETING TABLE COLUMNS =====")

print(fact_marketing.columns.tolist())

print("\n===== OVERALL MARKETING KPIs =====")

total_leads = fact_marketing["Leads"].sum()

total_conversions = fact_marketing["Conversions"].sum()

total_spend = fact_marketing["Spend"].sum()

overall_conversion_rate = (

    total_conversions / total_leads

) * 100

overall_cost_per_lead = (

    total_spend / total_leads

)

overall_cost_per_conversion = (
```

```

total_spend / total_conversions

)

print("Total Leads:", total_leads)

print("Total Conversions:", total_conversions)

print("Total Marketing Spend:", total_spend)

print("Overall Conversion Rate %:", overall_conversion_rate)

print("Overall Cost Per Lead:", overall_cost_per_lead)

print("Overall Cost Per Conversion:", overall_cost_per_conversion)

print("\n===== CAMPAIGN PERFORMANCE =====")

campaign_analysis = (

    fact_marketing

    .groupby("Campaign_Name", as_index=False)

    .agg(

        Total_Leads=("Leads", "sum"),

        Total_Conversions=("Conversions", "sum"),

        Total_Spend=("Spend", "sum")

    )

)

campaign_analysis["Conversion_Rate_%"] = (

    campaign_analysis["Total_Conversions"]

    / campaign_analysis["Total_Leads"]

) * 100

campaign_analysis["Cost_Per_Lead"] = (

    campaign_analysis["Total_Spend"]

    / campaign_analysis["Total_Leads"]

```

```

)

campaign_analysis["Cost_Per_Conversion"] = (

    campaign_analysis["Total_Spend"]

    / campaign_analysis["Total_Conversions"]

)

print(campaign_analysis)

print("\n===== TOP CAMPAIGNS BY CONVERSIONS =====")

top_conversion_campaigns = (

    campaign_analysis

    .sort_values(

        by="Total_Conversions",

        ascending=False

    )

    .head(10)

)

print(top_conversion_campaigns)

print("\n===== BEST CAMPAIGNS BY CONVERSION RATE =====")

best_conversion_rate_campaigns = (

    campaign_analysis

    .sort_values(

        by="Conversion_Rate_%",

        ascending=False

    )

    .head(10)

)

```

```

print(best_conversion_rate_campaigns)

print("\n===== MOST EFFICIENT CAMPAIGNS =====")

most_efficient_campaigns = (
    campaign_analysis
    .sort_values(
        by="Cost_Per_Conversion",
        ascending=True
    )
    .head(10)
)

print(most_efficient_campaigns)

print("\n===== HIGHEST SPENDING CAMPAIGNS =====")

highest_spending_campaigns = (
    campaign_analysis
    .sort_values(
        by="Total_Spend",
        ascending=False
    )
    .head(10)
)

print(highest_spending_campaigns)

print("\n===== CHANNEL PERFORMANCE =====")

channel_analysis = (
    fact_marketing
    .groupby("Channel", as_index=False)

```

```

.agg(

    Total_Leads=("Leads", "sum"),

    Total_Conversions=("Conversions", "sum"),

    Total_Spend=("Spend", "sum")

)

)

channel_analysis["Conversion_Rate_%"] = (

    channel_analysis["Total_Conversions"]

    / channel_analysis["Total_Leads"]

) * 100

channel_analysis["Cost_Per_Lead"] = (

    channel_analysis["Total_Spend"]

    / channel_analysis["Total_Leads"]

)

channel_analysis["Cost_Per_Conversion"] = (

    channel_analysis["Total_Spend"]

    / channel_analysis["Total_Conversions"]

)

print(channel_analysis)

print("\n===== BEST CHANNEL BY CONVERSIONS =====")

best_channel = (

    channel_analysis

    .sort_values(

        by="Total_Conversions",

        ascending=False

```

```

    )

    .iloc[0]

)

print(

    "Best Channel:",

    best_channel["Channel"]

)

print(

    "Total Conversions:",

    best_channel["Total_Conversions"]

)

print(

    "Conversion Rate %:",

    best_channel["Conversion_Rate_%"]

)

campaign_analysis.to_csv(

    "campaign_performance_analysis.csv",

    index=False

)

top_conversion_campaigns.to_csv(

    "top_campaigns_by_conversions.csv",

    index=False

)

best_conversion_rate_campaigns.to_csv(

    "best_campaigns_by_conversion_rate.csv",

```

```

    index=False
)

most_efficient_campaigns.to_csv(
    "most_efficient_campaigns.csv",
    index=False
)

channel_analysis.to_csv(
    "channel_performance_analysis.csv",
    index=False
)

print("\n✅ Marketing analysis files saved successfully!")

best_campaign_conversions = (
    campaign_analysis
    .sort_values(
        by="Total_Conversions",
        ascending=False
    )
    .iloc[0]
)

print("\n===== BEST CAMPAIGN BY CONVERSIONS =====")

print(
    "Campaign:",
    best_campaign_conversions["Campaign_Name"]
)

print(

```

```

    "Total Conversions:",

    best_campaign_conversions["Total_Conversions"]

)

print(

    "Conversion Rate %:",

    best_campaign_conversions["Conversion_Rate_%"]

)

best_campaign_rate = (

    campaign_analysis

    .sort_values(

        by="Conversion_Rate_%",

        ascending=False

    )

    .iloc[0]

)

print("\n===== BEST CAMPAIGN BY CONVERSION RATE =====")

print(

    "Campaign:",

    best_campaign_rate["Campaign_Name"]

)

print(

    "Conversion Rate %:",

    best_campaign_rate["Conversion_Rate_%"]

)

print(

```

```

    "Total Leads:",
    best_campaign_rate["Total_Leads"]
)
print(
    "Total Conversions:",
    best_campaign_rate["Total_Conversions"]
)
most_efficient_campaign = (
    campaign_analysis
    .sort_values(
        by="Cost_Per_Conversion",
        ascending=True
    )
    .iloc[0]
)
print("\n===== MOST COST-EFFICIENT CAMPAIGN =====")
print(
    "Campaign:",
    most_efficient_campaign["Campaign_Name"]
)
print(
    "Cost Per Conversion:",
    most_efficient_campaign["Cost_Per_Conversion"]
)
print(

```

```

    "Total Spend:",
    most_efficient_campaign["Total_Spend"]
)

print("\n=====")

print("MARKETING PERFORMANCE ANALYSIS COMPLETED!")

print("=====")

print("✅ Overall Marketing KPIs Calculated!")

print("✅ Campaign Performance Analyzed!")

print("✅ Conversion Rate Calculated!")

print("✅ Cost Per Lead Calculated!")

print("✅ Cost Per Conversion Calculated!")

print("✅ Channel Performance Analyzed!")

print("✅ Top Campaigns Identified!")

print("✅ Most Efficient Campaign Identified!")

print("✅ Analysis Files Saved Successfully!")

print("\n🎉 MARKETING ANALYSIS COMPLETED SUCCESSFULLY!")

```

5.9 CRM and pipeline analysis (crm.py)

The CRM and Pipeline Analysis module analyses customer relationship and sales pipeline data using the `crm_analysis.py` module. It helps evaluate deals at different CRM stages and understand the potential revenue available in the pipeline.

Source code :

```

import pandas as pd

fact_crm = pd.read_csv("fact_crm.csv")

print("CRM analytics table loaded successfully! ✅")

```

```

print("\n===== CRM TABLE COLUMNS =====")

print(fact_crm.columns.tolist())

print("\n===== OVERALL CRM KPIs =====")

total_deal_value = fact_crm["Deal_Value"].sum()

total_expected_revenue = fact_crm["Expected_Revenue"].sum()

average_deal_value = fact_crm["Deal_Value"].mean()

total_deals = fact_crm.shape[0]

average_probability = fact_crm["Probability"].mean() * 100

print("Total Deals:", total_deals)

print(
    "Total Deal Value:",
    total_deal_value
)

print(
    "Total Expected Revenue:",
    total_expected_revenue
)

print(
    "Average Deal Value:",
    average_deal_value
)

print(
    "Average Deal Probability %:",
    average_probability
)

```

```

print("\n===== CRM PERFORMANCE ANALYSIS =====")

crm_analysis = (

    fact_crm

    .groupby("Stage", as_index=False)

    .agg(

        Total_Deals=("Lead_ID", "nunique"),

        Total_Deal_Value=("Deal_Value", "sum"),

        Total_Expected_Revenue=("Expected_Revenue", "sum"),

        Average_Probability=("Probability", "mean")

    )

)

crm_analysis["Average_Probability_%"] = (

    crm_analysis["Average_Probability"] * 100

)

print(crm_analysis)

print("\n===== TOP CRM STAGES BY DEAL VALUE =====")

top_value_stages = (

    crm_analysis

    .sort_values(

        by="Total_Deal_Value",

        ascending=False

    )

)

print(top_value_stages)

print("\n===== TOP CRM STAGES BY EXPECTED REVENUE =====")

```

```

top_revenue_stages = (
    crm_analysis
    .sort_values(
        by="Total_Expected_Revenue",
        ascending=False
    )
)

print(top_revenue_stages)

print("\n===== HIGHEST PROBABILITY CRM STAGES =====")

highest_probability_stages = (
    crm_analysis
    .sort_values(
        by="Average_Probability_%",
        ascending=False
    )
)

print(highest_probability_stages)

print("\n===== TOP 10 DEALS BY DEAL VALUE =====")

top_deals = (
    fact_crm
    .sort_values(
        by="Deal_Value",
        ascending=False
    )
    .head(10)
)

```

```

)

print(top_deals)

print("\n===== TOP 10 DEALS BY EXPECTED REVENUE =====")

top_expected_revenue_deals = (

    fact_crm

    .sort_values(

        by="Expected_Revenue",

        ascending=False

    )

    .head(10)

)

print(top_expected_revenue_deals)

crm_analysis.to_csv(

    "crm_stage_analysis.csv",

    index=False

)

top_deals.to_csv(

    "top_10_crm_deals.csv",

    index=False

)

top_expected_revenue_deals.to_csv(

    "top_10_expected_revenue_deals.csv",

    index=False

)

print("\n✅ CRM analysis files saved successfully!")

```

```

highest_value_stage = (
    crm_analysis
    .sort_values(
        by="Total_Deal_Value",
        ascending=False
    )
    .iloc[0]
)

print("\n===== HIGHEST VALUE CRM STAGE =====")

print(
    "Stage:",
    highest_value_stage["Stage"]
)

print(
    "Total Deal Value:",
    highest_value_stage["Total_Deal_Value"]
)

print(
    "Expected Revenue:",
    highest_value_stage["Total_Expected_Revenue"]
)

highest_revenue_stage = (
    crm_analysis
    .sort_values(
        by="Total_Expected_Revenue",

```

```

        ascending=False

    )

    .iloc[0]

)

print("\n===== HIGHEST EXPECTED REVENUE STAGE =====")

print(

    "Stage:",

    highest_revenue_stage["Stage"]

)

print(

    "Expected Revenue:",

    highest_revenue_stage["Total_Expected_Revenue"]

)

print(

    "Average Probability %:",

    highest_revenue_stage["Average_Probability_%"]

)

highest_value_deal = (

    fact_crm

    .sort_values(

        by="Deal_Value",

        ascending=False

    )

    .iloc[0]

)

```

```
print("\n===== HIGHEST VALUE DEAL =====")

print(
    "Lead ID:",
    highest_value_deal["Lead_ID"]
)

print(
    "Deal Value:",
    highest_value_deal["Deal_Value"]
)

print(
    "Expected Revenue:",
    highest_value_deal["Expected_Revenue"]
)

print(
    "Probability:",
    highest_value_deal["Probability"] * 100
)

print("CRM PERFORMANCE ANALYSIS COMPLETED!")

print("✅ Overall CRM KPIs Calculated!")

print("✅ CRM Stage Performance Analyzed!")

print("✅ Deal Value Analysis Completed!")

print("✅ Expected Revenue Analysis Completed!")

print("✅ Probability Analysis Completed!")

print("✅ Top CRM Deals Identified!")
```

```
print("✅ CRM Analysis Files Saved Successfully!")
```

```
print("\n🎉 CRM ANALYSIS COMPLETED SUCCESSFULLY!")
```

5.10 KPI Calculation and Analysis (kpi_analysis.py)

The KPI Calculation and Analysis module is implemented using the kpi_analysis.py file. It calculates important business performance indicators from the processed SalesSight 360 datasets. The module provides a summarized view of sales, profit, marketing, CRM, and target performance.

Source code:

```
import pandas as pd

fact_orders = pd.read_csv("fact_orders.csv")

fact_payments = pd.read_csv("fact_payments.csv")

fact_marketing = pd.read_csv("fact_marketing.csv")

fact_crm = pd.read_csv("fact_crm.csv")

fact_target = pd.read_csv("fact_target.csv")

print("All analytics tables loaded successfully! ✅")

total_sales = fact_orders["Sales"].sum()

total_profit = fact_orders["Profit"].sum()

total_orders = fact_orders["Order_ID"].nunique()

average_order_value = total_sales / total_orders

print("Total Sales:", total_sales)

print("Total Profit:", total_profit)

print("Total Orders:", total_orders)

print("Average Order Value:", average_order_value)

overall_profit_margin = (

    total_profit / total_sales

) * 100

print("Overall Profit Margin %:", overall_profit_margin)
```

```

print("\n===== DISCOUNT KPIs =====")

total_discount = fact_orders["Discount_Amount"].sum()

total_net_sales = fact_orders["Net_Sales"].sum()

print("Total Discount Amount:", total_discount)

print("Total Net Sales:", total_net_sales)

total_payment_amount = fact_payments["Amount"].sum()

print("Total Payment Amount:", total_payment_amount)

total_leads = fact_marketing["Leads"].sum()

total_conversions = fact_marketing["Conversions"].sum()

total_marketing_spend = fact_marketing["Spend"].sum()

overall_conversion_rate = (

    total_conversions / total_leads

) * 100

overall_cost_per_lead = (

    total_marketing_spend / total_leads

)

overall_cost_per_conversion = (

    total_marketing_spend / total_conversions

)

print("Total Leads:", total_leads)

print("Total Conversions:", total_conversions)

print("Total Marketing Spend:", total_marketing_spend)

print("Overall Conversion Rate %:", overall_conversion_rate)

print("Overall Cost Per Lead:", overall_cost_per_lead)

print("Overall Cost Per Conversion:", overall_cost_per_conversion)

```

```
total_deal_value = fact_crm["Deal_Value"].sum()

total_expected_revenue = fact_crm["Expected_Revenue"].sum()

print("Total Deal Value:", total_deal_value)

print("Total Expected Revenue:", total_expected_revenue)

print("Target Table Rows:", len(fact_target))

print("\nTarget Columns:")

print(fact_target.columns.tolist())

print("Total Sales:", total_sales)

print("Total Profit:", total_profit)

print("Total Orders:", total_orders)

print("Average Order Value:", average_order_value)

print("Overall Profit Margin %:", overall_profit_margin)

print("Total Net Sales:", total_net_sales)

print("Total Payment Amount:", total_payment_amount)

print("Total Leads:", total_leads)

print("Total Conversions:", total_conversions)

print("Overall Conversion Rate %:", overall_conversion_rate)

print("Total Deal Value:", total_deal_value)

print("Total Expected Revenue:", total_expected_revenue)

print("\n✅ KPI ANALYSIS COMPLETED SUCCESSFULLY! 🎉")
```

CHAPTER-6

TESTING

6.1 Introduction

Testing is an important phase of the SalesSight 360 – Multi-Channel Revenue Analytics Platform development process. The main purpose of testing is to verify that the implemented modules work correctly and that the generated analytical results are accurate, consistent, and reliable. Testing was performed on different stages of the system, including data loading, data cleaning, transformation, validation, data modelling, analysis, KPI calculation, and Power BI visualization.

6.1 Testing Methodology

The SalesSight 360 project follows a systematic testing methodology to verify the correctness and reliability of the data analytics pipeline. Testing is performed at different stages, from data processing to final Power BI visualization.

The main testing methods used are:

Data Validation Testing – Checks missing, invalid, duplicate, and inconsistent data.

ETL Testing – Verifies data loading, cleaning, transformation, and output generation through etl.py.

Data Model Testing – Verifies relationships between fact and dimension tables.

Module Testing – Tests individual Python modules such as customer, product, marketing, CRM, target, and KPI analysis.

KPI Testing – Verifies calculated metrics such as sales, profit, conversion rate, variance, and target achievement.

Integration Testing – Ensures that outputs from Python modules are correctly used for Power BI reporting.

Dashboard Testing – Checks Power BI visuals, slicers, filters, navigation, and displayed values.

6.2 Testing Summary

Testing of the SalesSight 360 project was performed to verify the correctness, reliability, and consistency of the complete analytics workflow. The major Python modules, data relationships, analytical calculations, and Power BI dashboard functions were tested.

The testing confirmed that:

Raw datasets were loaded and processed successfully.

Data cleaning and transformation worked correctly.

Customer, product, and order relationships were validated.

0 invalid Customer_ID, Product_ID, and Order_ID references were found.

Fact and dimension tables were generated successfully.

Customer, product, sales trend, marketing, CRM, and target analyses produced the expected outputs.

KPI calculations were generated successfully.

Target versus actual analysis worked correctly.

Power BI dashboards, slicers, filters, and navigation functioned properly.

Overall, the testing results were successful, confirming that SalesSight 360 provides consistent, accurate, and reliable analytical outputs from raw business data to final Power BI dashboards.

CHAPTER 7

LIMITATIONS OF THE PROJECT

7.1 Introduction

Although **SalesSight 360 – Multi-Channel Revenue Analytics Platform** provides an integrated solution for business data analysis and visualization, the project has certain limitations. These limitations mainly arise from the nature of the available datasets, the technologies used, and the scope of the project. The system is designed primarily for **analytical and reporting purposes**. It processes existing business data through Python and presents the resulting insights through Power BI dashboards. Therefore, some advanced features such as real-time data integration, automated deployment, and predictive analytics are outside the current scope of the project. Understanding these limitations is important because it helps identify areas where the system can be improved and extended in future versions.

7.1 Limitations of the Project

Although SalesSight 360 provides an integrated platform for business analytics, the current version has some limitations:

Static Data Sources – The project mainly uses CSV/Excel datasets, so data is not automatically received from live business systems.

No Real-Time Processing – The dashboards depend on processed data and do not currently provide real-time updates.

Limited Dataset Size – The project has been developed and tested using a specific set of datasets, so performance with very large enterprise-level data has not been evaluated.

Manual Data Refresh – Updated source data may require the ETL process and Power BI data to be refreshed before new information appears in the dashboards.

Limited Predictive Analytics – The current system mainly focuses on descriptive and diagnostic analytics.

Advanced machine-learning-based prediction is not a major part of the current implementation.

Dependency on Data Quality – The accuracy of the analysis depends on the quality, completeness, and consistency of the source data.

Limited External Integration – The current system does not directly integrate with live CRM, ERP, payment, or marketing platforms.

Deployment Scope – The project is primarily developed as an academic/business intelligence prototype and is not currently implemented as a fully deployed enterprise application.

These limitations provide opportunities for future improvements such as real-time data integration, automated refresh, predictive analytics, cloud deployment, and integration with live business systems

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

CONCLUSION

SalesSight 360 successfully integrates sales, marketing, CRM, customer, and payment data to provide meaningful business insights. The project uses Python, Pandas, ETL, data modelling, and Power BI to transform raw data into interactive dashboards. Overall, it supports data-driven decision-making and better understanding of sales performance.

8.1 Future Scope

The **SalesSight 360** project can be further enhanced with advanced technologies and features to make it more powerful and suitable for real-world business environments.

1. **Real-Time Data Integration** – Integration with live CRM, ERP, payment, and marketing systems can provide continuously updated business information.
2. **Automated Data Refresh** – The ETL process can be automated so that new data is processed and dashboards are refreshed without manual intervention.
3. **Predictive Analytics** – Machine learning can be introduced to predict future sales, customer behaviour, demand, and revenue.
4. **Advanced Customer Analytics** – Customer segmentation and customer lifetime value analysis can help identify different customer groups and their potential value.
5. **AI-Based Business Insights** – AI can automatically identify unusual trends, performance gaps, and important business patterns.
6. **Cloud Deployment** – The platform can be deployed on cloud services to improve accessibility, scalability, and collaboration.
7. **Advanced Power BI Features** – More advanced dashboards, drill-through reports, forecasting, and natural-language analysis can be incorporated.

8. **Scalability** – The system can be extended to handle larger datasets and additional business domains.
9. **Automated Alerts** – Notifications can be generated when sales fall below targets, conversion rates change significantly, or other important business conditions occur.
10. **Mobile Access** – Mobile-friendly dashboards can allow managers to monitor important KPIs from anywhere.

Overall, the future scope of SalesSight 360 is to develop it into a **scalable, automated, AI-enabled, and real-time business intelligence platform** that can provide deeper insights and support faster data-driven decision-making.

Applications of the Project

The SalesSight 360 – Multi-Channel Revenue Analytics Platform can be applied in different business areas to analyse performance and support data-driven decision-making.

1. Sales Performance Analysis – Helps organizations monitor sales, profit, orders, and monthly sales trends.
2. Customer Analysis – Helps identify high-performing customers and understand their contribution to overall revenue.
3. Product Performance Analysis – Identifies products with high sales, quantity, and profit contribution.
4. Marketing Analysis – Helps compare marketing campaigns and channels using leads, conversions, spending, and conversion rates.
5. CRM and Pipeline Management – Helps monitor deals, sales stages, deal values, and expected revenue.
6. Target Performance Monitoring – Compares actual sales with predefined targets and identifies performance gaps.
7. Management Reporting – Provides interactive Power BI dashboards that allow managers to monitor important KPIs in a single platform.
8. Business Decision-Making – Provides meaningful insights that can help organizations identify trends, evaluate performance, and make better data-driven decisions.

8.2 Benefits of Future Enhancements

The proposed future enhancements of SalesSight 360 can provide several benefits and make the system more effective for real-world business use.

- **Real-Time Decision Making:** Live data integration can provide updated business information and support faster decisions.
- **Improved Accuracy:** Automated ETL and data-refresh processes can reduce manual errors.
- **Better Sales Forecasting:** Predictive analytics can help estimate future sales and revenue.
- **Deeper Customer Insights:** Advanced customer analytics can help understand customer behaviour and value.
- **Improved Marketing Efficiency:** AI-based analysis can identify effective campaigns and channels.
- **Faster Performance Monitoring:** Automated alerts can highlight important changes or performance gaps.
- **Scalability:** Cloud-based deployment can allow the system to handle larger datasets and more users.
- **Better Accessibility:** Mobile and cloud dashboards can make business insights available from different locations.
- **Reduced Manual Effort:** Automation can reduce repetitive data-processing and reporting tasks.
- **Stronger Business Intelligence:** AI and advanced analytics can provide more meaningful and actionable insights.

8.3Summary

SalesSight 360 – Multi-Channel Revenue Analytics Platform is a data analytics and business intelligence solution developed to integrate and analyse data from multiple business functions, including sales, customers, products, payments, marketing, CRM, and sales targets.

The project uses **Python and Pandas** for data loading, cleaning, transformation, validation, data modelling, and analytical processing. Different Python modules perform customer, product, sales trend, marketing, CRM, target, and KPI analysis. The processed data is then used in **Microsoft Power BI** to create interactive dashboards and reports. Although the current system has limitations such as static datasets and the absence of real-time processing, it provides a strong foundation for future enhancements including **real-time integration, automated refresh, predictive analytics, AI-based insights, cloud deployment, and automated alerts.**

CHAPTER 9

BIBLIOGRAPHY

The development of the **SalesSight 360 – Multi-Channel Revenue Analytics Platform** involved the study of data analytics concepts, ETL processes, data preprocessing, data modelling, business intelligence, KPI analysis, and dashboard development. Various official technical resources, software documentation, and reference books were consulted during the implementation and understanding of the technologies used in this project. The following sources provided useful guidance throughout the development process.

Software and Development Tools

- Python 3.x
- MicroSoft PowerBI
- Pandas
- Visual Studio Code